Transacciones y programación SQL

`COMMIT`, `ROLLBACK`, funciones definidas por el usuario, variables e `IF`

Semana 13 - Sesión 26

Rel Guzman



Logro de la sesión

Al finalizar, podrás registrar una compra con una **transacción**, que se guarda **entera o nada** con `COMMIT` y `ROLLBACK`, y escribir **funciones** con **variables** e **IF**, en MySQL y SQL Server.

Revisión: ¿qué hace `OVER`?

Los 3 pagos de hoy y su promedio, de dos formas:

Sin OVER

```
SELECT AVG(monto) AS promedio
FROM pago WHERE id >= 6;
```

resultado

promedio
35.000000

Junta las filas: queda **1**.

Con OVER

```
SELECT id, monto, AVG(monto) OVER () AS promedio
FROM pago WHERE id >= 6 ORDER BY id;
```

resultado

id	monto	promedio
6	15.00	35.000000
7	35.00	35.000000
8	55.00	35.000000

No junta: quedan las **3**, cada una con el promedio al lado.

`OVER`

Calcula sobre varias filas **sin juntarlas**.

Revisión: lo que va dentro de `OVER`

Se escribe	Calcula sobre
`OVER ()`	todas las filas
`OVER (PARTITION BY metodo)`	cada grupo por separado
`OVER (ORDER BY id)`	las filas en orden, sumando lo anterior

```sql
SELECT id, monto,
   SUM(monto) OVER (ORDER BY id) AS acumulado
FROM pago WHERE id >= 6 ORDER BY id;
```

resultado

id	monto	acumulado
6	15.00	15.00
7	35.00	50.00
8	55.00	105.00

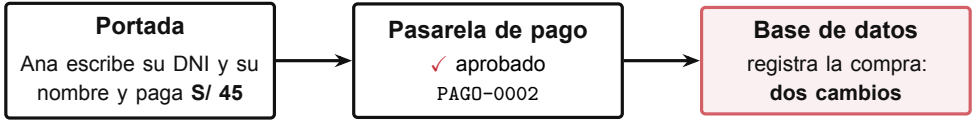
15, luego 15 + 35, luego 50 + 55

También van con `OVER`: `ROW_NUMBER`, `RANK`, `DENSE_RANK`, `LAG` y `LEAD`.

Transacciones

Comprar una entrada: dos cambios

La feria **Fogón 2026** vende una sola entrada, a S/ 45, con **500 entradas disponibles**. Cada persona compra **1 entrada**. Ana compra la suya:



1 Descontar **1 entrada disponible**

`entrada`

nombre	disponibles
Entrada Fogón 2026	500 → 499

2 Guardar la **compra**

`compra`

dni	nombre	precio	referencia_pago
45678912	Ana Quispe	45.00	PAGO-0002

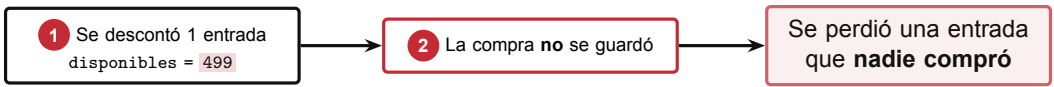
Idea clave

Los dos cambios van **juntos**: o se hacen los dos, o ninguno.

¿Y si el segundo cambio falla?

Luis **ya compró** su entrada. Intenta comprar otra con el mismo DNI. Cada persona compra solo una: `dni` es `UNIQUE`.

```
UPDATE entrada SET disponibles = disponibles - 1 WHERE id = 1;                ✓
INSERT INTO compra (dni, nombre, precio, referencia_pago)
VALUES ('70123456', 'Luis Rojas', 45.00, 'PAGO-0003');                        ×
-- ERROR 1062 (23000): Duplicate entry '70123456' for key 'compra.dni'
```



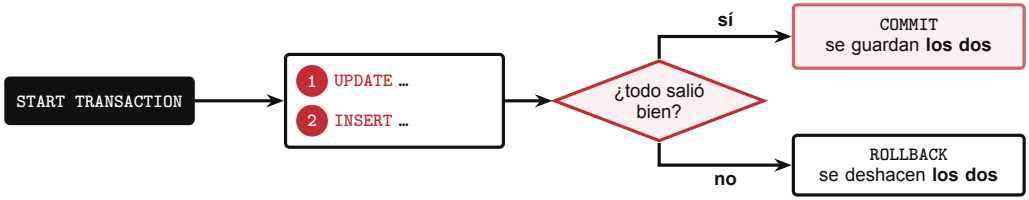
Ojo con esto

Cada instrucción se guarda **apenas termina**. El `UPDATE` ya quedó guardado cuando falló el `INSERT`.

Transacción: todo o nada

Transacción

Un grupo de cambios que se guardan **todos juntos** o **ninguno**.



	MySQL	SQL Server
Empezar	`START TRANSACTION;`	`BEGIN TRANSACTION;`
Guardar todo	`COMMIT;`	`COMMIT;`
Deshacer todo	`ROLLBACK;`	`ROLLBACK;`

La compra de Ana: COMMIT

```
START TRANSACTION;
UPDATE entrada SET disponibles = disponibles - 1 WHERE id = 1;
INSERT INTO compra (dni, nombre, precio, referencia_pago)
VALUES ('45678912', 'Ana Quispe', 45.00, 'PAGO-0002');
COMMIT;
```

```
SELECT disponibles FROM entrada;
```

resultado

disponibles
499

```
SELECT dni, nombre, referencia_pago FROM compra ORDER BY id;
```

resultado

dni	nombre	referencia_pago
70123456	Luis Rojas	PAGO-0001
45678912	Ana Quispe	PAGO-0002

COMMIT

Todo salió bien: los **dos cambios** quedan guardados a la vez.

Luis otra vez: ROLLBACK

```
START TRANSACTION;
UPDATE entrada SET disponibles = disponibles - 1 WHERE id = 1;
SELECT disponibles FROM entrada;                                  1

INSERT INTO compra (dni, nombre, precio, referencia_pago)
VALUES ('70123456', 'Luis Rojas', 45.00, 'PAGO-0003');
-- ERROR 1062: Duplicate entry '70123456' for key 'compra.dni'

ROLLBACK;
SELECT disponibles FROM entrada;                                  2
```



ROLLBACK

Algo falló: se deshace **todo**. La entrada vuelve a estar disponible y no queda ninguna compra a medias.

Práctica 1 Una compra con transacción

En cada gestor ejecuta **tres scripts, en orden**. Están en `ejemplos/practicas/mysql/` y `ejemplos/practicas/sqlserver/`:

Paso	Script	Qué hace
1	`practica_1_modelo_fisico.sql`	crea la base `feria` y sus tablas
2	`practica_1_registros.sql`	carga la entrada (500 disponibles) y la compra de Luis
3	`practica_1_transaccion.sql`	la compra de Ana y el intento de Luis

Comprueba las entradas `disponibles`:

Antes	Compra de Ana	Intento de Luis	Después del `ROLLBACK`
500	499	498 (pendiente)	499

Ojo con esto

El `INSERT` de Luis falla a propósito. Workbench se detiene ahí: selecciona las líneas que siguen y ejecútalas. Para repetir la práctica, vuelve a ejecutar los pasos 1 y 2.

Transacciones en MySQL, SQL Server y SQLite

	MySQL	SQL Server	SQLite
Empezar	`START TRANSACTION;`	`BEGIN TRANSACTION;`	`BEGIN TRANSACTION;`
Guardar todo	`COMMIT;`	`COMMIT;`	`COMMIT;`
Deshacer todo	`ROLLBACK;`	`ROLLBACK;`	`ROLLBACK;`
Sin transacción	cada instrucción se guarda **apenas termina**		
¿Un error la deshace?	**no**: tú decides `ROLLBACK`		
La compra de Ana y Luis	mismo resultado: 500 → 499 → 498 → 499		
`CREATE TABLE` **dentro**	se guarda al instante: `ROLLBACK` no la quita	`ROLLBACK` la quita	`ROLLBACK` la quita

¿Funcionan igual?

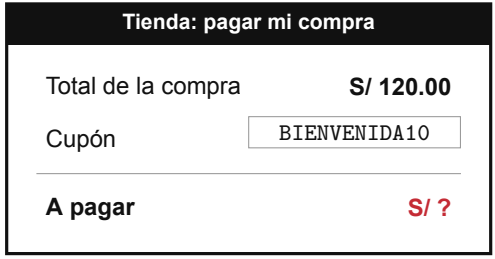
Sí con `INSERT`, `UPDATE` y `DELETE`: todo o nada. Solo cambia cómo se empieza, y en MySQL no se crean tablas dentro de una transacción.

Programación en SQL

Caso de estudio: sistema de cupones de descuento

Una tienda en línea lanza **códigos de descuento**.

- El **responsable de promociones** registra los cupones y los activa o desactiva.
- El **cliente** escribe el código al pagar.
- Un cupón descuenta un **porcentaje** o un **monto fijo** en soles.
- Tiene fecha de inicio y, si se quiere, fecha de fin.



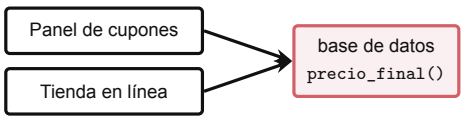
cupon

id	codigo	tipo_descuento	valor_descuento	estado	vigente_desde	vigente_hasta
1	BIENVENIDA10	porcentaje	10.00	activo	2026-01-01	NULL
2	NOVIEMBRE20	monto_fijo	20.00	activo	2026-11-01	2026-12-01
3	VERANO15	porcentaje	15.00	activo	2026-01-01	2026-04-01
4	NAVIDAD25	porcentaje	25.00	activo	2026-12-15	2026-12-26
5	PRUEBA50	monto_fijo	50.00	inactivo	2026-10-01	NULL

¿Cuánto paga el cliente? La fórmula va en la base

Una compra de **S/ 120**. El cálculo depende del tipo de cupón:

Cupón	Tipo	Cálculo	Paga
`BIENVENIDA10`	porcentaje: 10	$120 - 120 \times 10/100$	**S/ 108.00**
`NOVIEMBRE20`	monto fijo: 20	$120 - 20$	**S/ 100.00**



Los **dos sistemas** necesitan el mismo cálculo. Si lo escribe cada uno, el panel puede mostrar un precio y la tienda cobrar otro.

Se escribe **una vez**, en la base, como **función**, y todos la usan.

Función definida por el usuario

Una función que **tú escribes** con `CREATE FUNCTION`. Queda **guardada en la base** y se usa en un `SELECT`, igual que `ROUND` o `UPPER`.

Una función en MySQL y en SQL Server

El caso más simple: un cupón de **porcentaje**.

MySQL

```mysql
CREATE FUNCTION aplicar_porcentaje(
  p_precio DECIMAL(10,2),
  p_porcentaje DECIMAL(5,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
RETURN p_precio - p_precio * p_porcentaje / 100;

SELECT aplicar_porcentaje(120, 10); -- 108.00
```

SQL Server

```sql
CREATE OR ALTER FUNCTION dbo.aplicar_porcentaje(
  @precio DECIMAL(10,2),
  @porcentaje DECIMAL(5,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
  RETURN @precio - @precio * @porcentaje / 100;
END;
GO

SELECT dbo.aplicar_porcentaje(120, 10); -- 108.00
```

	MySQL	SQL Server
Parámetros	`p_precio` (`p_`: costumbre)	`@precio`: la `@` es obligatoria
Cuerpo	`RETURN …;`	entre `AS BEGIN` y `END`
Para usarla	`aplicar_porcentaje(…)`	`dbo.aplicar_porcentaje(…)`

Usarla sobre las filas de una tabla

En el panel: con una compra de S/ 120, ¿cuánto pagaría el cliente con cada cupón de porcentaje?

```
SELECT codigo, valor_descuento,
   aplicar_porcentaje(120, valor_descuento)
     AS a_pagar
FROM cupon
WHERE tipo_descuento = 'porcentaje'
ORDER BY id;
```

resultado

codigo	valor_descuento	a_pagar
BIENVENIDA10	10.00	108.00
VERANO15	15.00	102.00
NAVIDAD25	25.00	90.00

Idea clave

La función se ejecuta **una vez por cada fila**: cada vez recibe el `valor_descuento` de esa fila. En SQL Server se escribe `dbo.aplicar_porcentaje`.

Un cupón de monto fijo necesita variables e IF

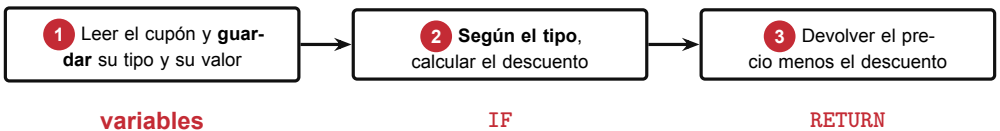
`NOVIEMBRE20` descuenta **S/ 20**, no el 20 %. Con la función de antes, el cálculo sale mal:

```
SELECT aplicar_porcentaje(120, 20);
```

resultado

`96.00` debía ser **100.00**

La función `precio_final(precio, cupon)` tiene que hacer **tres pasos**:



Variables: guardar un valor para usarlo después

Variable

Un **nombre** que guarda **un valor** mientras se ejecuta tu código. Se le da un valor con `SET`.

MySQL

```
SET @precio = 120;
SET @descuento = 20;
SELECT @precio - @descuento AS a_pagar;
-- 100
```

Guardar lo que devuelve un `SELECT`:

```
SELECT valor_descuento INTO @descuento
FROM cupon WHERE codigo = 'NOVIEMBRE20';
```

SQL Server

```
DECLARE @precio DECIMAL(10,2) = 120;
DECLARE @descuento DECIMAL(10,2) = 20;
SELECT @precio - @descuento AS a_pagar;
-- 100.00
```

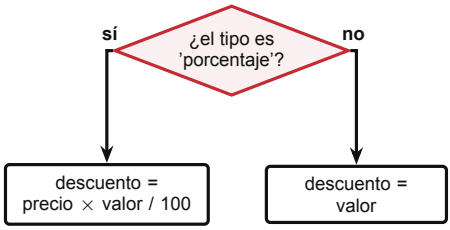
Guardar lo que devuelve un `SELECT`:

```
SELECT @descuento = valor_descuento
FROM cupon WHERE codigo = 'NOVIEMBRE20';
```

Ojo con esto

Dentro de una función de MySQL, las variables se crean con `DECLARE` y van **sin** `@`: `DECLARE v_tipo VARCHAR(10);`

IF: elegir un camino



Se ejecuta **solo uno** de los dos caminos.

MySQL

```
IF v_tipo = 'porcentaje' THEN
  SET v_descuento = p_precio * v_valor / 100;
ELSE
  SET v_descuento = v_valor;
END IF;
```

SQL Server

```
IF @tipo = 'porcentaje'
  SET @descuento = @precio * @valor / 100;
ELSE
  SET @descuento = @valor;
```

En palabras simples

En SQL Server no hay `THEN` ni `END IF`. Si un camino tiene **varias** instrucciones, van entre `BEGIN` y `END`.

Todo junto: `precio_final` en MySQL

```
DELIMITER $$
CREATE FUNCTION precio_final(p_precio DECIMAL(10,2),
                             p_cupon_id INT)
RETURNS DECIMAL(10,2)
READS SQL DATA
BEGIN
  DECLARE v_tipo VARCHAR(10);
  DECLARE v_valor DECIMAL(10,2);
  DECLARE v_descuento DECIMAL(10,2);
  SELECT tipo_descuento, valor_descuento INTO v_tipo, v_valor
  FROM cupon WHERE id = p_cupon_id;                          1
  IF v_tipo = 'porcentaje' THEN
    SET v_descuento = p_precio * v_valor / 100;
  ELSE
    SET v_descuento = v_valor;
  END IF;                                                    2
  IF v_descuento > p_precio THEN
    SET v_descuento = p_precio;
  END IF;
  RETURN p_precio - v_descuento;                             3
END $$
DELIMITER ;
```

`BEGIN ... END`: agrupa varias instrucciones, cada una con su `;`.

`DELIMITER $$`: le dice a MySQL que la función termina en `$$`, no en el primer `;`.

`READS SQL DATA`: la función lee una tabla.

El segundo `IF` evita que se pague menos de 0.

```
SELECT precio_final(120, 1), -- 108.00
       precio_final(120, 2), -- 100.00
       precio_final(30, 5);   --   0.00
```

Cupones 1, 2 y 5. Con `PRUEBA50` el descuento (50) es mayor que el precio (30): paga 0.

IF con varios caminos: ELSEIF

¿En qué situación está cada cupón? Se revisa **en orden**: gana la primera condición que se cumple.

MySQL

```
IF v_estado = 'inactivo' THEN
  SET v_situacion = 'inactivo';
ELSEIF p_fecha < v_desde THEN
  SET v_situacion = 'por empezar';
ELSEIF v_hasta IS NOT NULL
    AND p_fecha >= v_hasta THEN
  SET v_situacion = 'vencido';
ELSE
  SET v_situacion = 'vigente';
END IF;
```

SQL Server

```
IF @estado = 'inactivo' …
ELSE IF @fecha < @desde …
ELSE IF @hasta IS NOT NULL AND …
ELSE …
```

```
SELECT codigo,
  situacion_cupon(id, '2026-11-10') AS situacion
FROM cupon ORDER BY id;
```

resultado

codigo	situacion
BIENVENIDA10	vigente
NOVIEMBRE20	vigente
VERANO15	vencido
NAVIDAD25	por empezar
PRUEBA50	inactivo

La fecha es fija para que todos veamos lo mismo.
En la aplicación se usa la fecha de hoy.

Práctica 2 La base de cupones y sus funciones

1. Crea la base `cupones`: dos scripts, en orden. Están en `ejemplos/practicas/`:

Gestor	1. Modelo físico (tablas)	2. Registros (datos)
MySQL (Workbench)	`mysql/practica_2_modelo_fisico.sql`	`mysql/practica_2_registros.sql`
SQL Server (SSMS)	`sqlserver/practica_2_modelo_fisico.sql`	`sqlserver/practica_2_registros.sql`

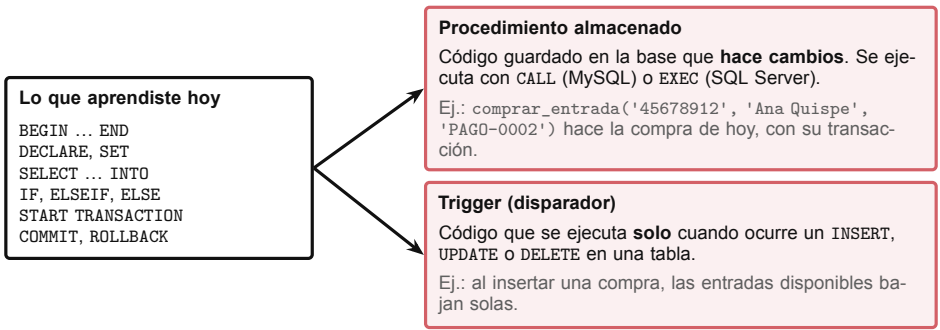
2. Crea las funciones y pruébalas. Los scripts están en `ejemplos/2_cupones/`, en este orden:

Gestor	1. Primera función	2. Variables e `IF`
MySQL	`1_funciones_mysql.sql`	`2_variables_if_mysql.sql`
SQL Server	`1_funciones_sqlserver.sql`	`2_variables_if_sqlserver.sql`

3. Comprueba que al final del segundo script salgan solo los cupones vigentes: **BIENVENIDA10** paga **108.00** y **NOVIEMBRE20** paga **100.00**.

Lo que viene: procedimientos almacenados y triggers

La sintaxis de hoy no sirve solo para funciones. Con ella se programan las dos cosas que vienen:



Idea clave

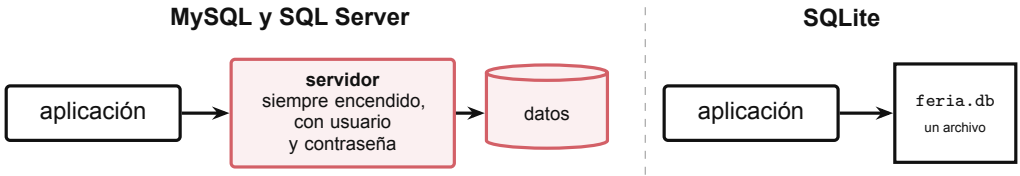
Una **función** calcula y devuelve un valor. Un **procedimiento** hace cambios cuando lo llamas. Un **trigger** hace cambios cuando ocurre algo en una tabla.

Universidad Tecnológica del Perú

SQLite

SQLite: una base de datos en un archivo

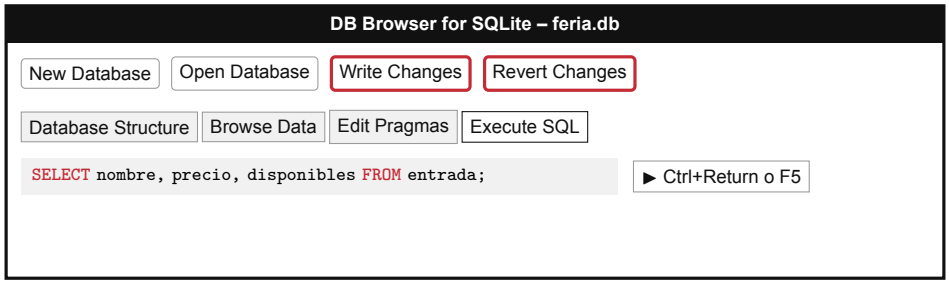
La aplicación de la feria Fogón 2026 guarda sus datos en **SQLite**. No hay un servidor: toda la base es **un archivo**.



- No se instala ningún servidor.
- La base se copia o se envía como cualquier archivo.
- Se usa en celulares, navegadores y aplicaciones pequeñas.
- El SQL es casi el mismo: `SELECT`, `INSERT`, `BEGIN TRANSACTION`…

DB Browser for SQLite

Un programa gratuito para abrir un archivo `.db`, ver sus tablas y ejecutar SQL. Es como Workbench o SSMS, pero para SQLite.



Database Structure	las tablas y sus columnas
Browse Data	las filas de una tabla, para verlas o editarlas
Execute SQL	escribir SQL y ejecutarlo

DB Browser trabaja con transacciones

En DB Browser, lo que cambias queda **pendiente** hasta que pulsas **Write Changes**:



Ojo con esto

Si cierras DB Browser sin pulsar **Write Changes**, te pregunta si quieres guardar. Y mientras no guardes, la aplicación de la feria **no ve** tus cambios.

Preguntas



Cierre

Transacción	Varias instrucciones que se guardan **todas o ninguna**.
Empezar	MySQL: `START TRANSACTION`; SQL Server y SQLite: `BEGIN TRANSACTION`.
`COMMIT`	Confirma y guarda los cambios.
`ROLLBACK`	Deshace los cambios: todo vuelve a como estaba.
Función del usuario	Tu propio cálculo, guardado en la base con `CREATE FUNCTION`.
Variable	Un nombre que guarda un valor: `DECLARE`, `SET`, `SELECT … INTO`.
`IF`	Elige un camino. MySQL: `ELSEIF` y `END IF`; SQL Server: `ELSE IF`.
SQLite	Una base de datos en un archivo. Se abre con DB Browser for SQLite.

Referencias

- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2014). *Fundamentos de bases de datos* (6.ª ed.). McGraw-Hill.
- Oracle. MySQL Reference Manual: *START TRANSACTION, COMMIT, and ROLLBACK Statements*.
- Oracle. MySQL Reference Manual: *CREATE PROCEDURE and CREATE FUNCTION* y *IF Statement*.
- Microsoft. Transact-SQL: *BEGIN TRANSACTION* y *ROLLBACK TRANSACTION*.
- Microsoft. Transact-SQL: *CREATE FUNCTION* y *IF...ELSE*.
- SQLite. *Transaction*. DB Browser for SQLite: *sqlitebrowser.org*.



UTP Universidad Tecnológica del Perú